

UNIVERSIDAD
DE GRANADA

Práctica 1

Problema del portfolio

Pablo Linari Pérez

DNI: 75571079W
Grupo de prácticas: 3 (Martes 15:30-17:30)
Curso: 25/26
Correo: pablolinari@correo.ugr.es

Índice de contenidos

1 Descripción del problema 3
2 Algoritmos empleados y estructura del problema 4
2.1 Estructura del problema 4
2.2 Representación de las soluciones 4
2.3 Funciones comunes usadas por los algoritmos 4
3 Estructura de los algoritmos 7
3.1 Random Search 7
3.2 Greedy 7
3.3 Local Search 8
3.4 Metodología 8
4 Conclusión 9

1 Descripción del problema

El problema que se ha escogido es el problema del portfolio, en el cual se nos da una lista de empresas de las cuales sabemos su retorno obtenido diario desde principios de 2015 hasta finales de 2025. Con estos datos necesitamos elaborar una solución indicando qué porcentaje de nuestra cartera deberíamos haber invertido en cada empresa para maximizar el beneficio de las inversiones.

En este contexto, el problema no solo busca el máximo retorno, sino que también suele considera la correlaicion entre activos teniendo esto en cuenta en las ecuaciones que se usan para maximizar el beneficio.

De esta manera podemos ajustar el codigo ajustando el parametro λ de la ecuacion a maximizar para asi tener mayor o menor aversion al riesgo.

2 Algoritmos empleados y estructura del problema

2.1 Estructura del problema

Para modelar el problema del portfolio se ha usado la clase Pportfolio la cual es hija de la clase Problem que viene ya predefinnida con varios métodos virtuales , los cuales se implementan en la clase hija. Para representar el problema usando estas clases se tienen en cuenta los siguientes atributos:

- **days** : número entero de días de los que se tienen datos.
- **ncompany** : número entero de empresas de las que se tienen datos.
- **low** : mínimo porcentaje de inversión en una empresa (double).
- **high** : máximo porcentaje de inversión en una empresa (double).
- **lambda** : parámetro entero usado en el fitness.
- **data** : matriz de la libreria Eigen con valores double que contienen los beneficios diarios de cada empresa, en las columnas se representan las empresas y en las filas los dias.
- **covarianceMat** : matriz de la libreria Eigen con valores double que contiene la covarianza de las empresas.
- **ERROR** : Error permitido en la suma total del vector solución (double).

Podemos observar que se ha optado por usar la libreria Eigen de C++ para poder manejar las operaciones de matrices y vectores como la suma total , la media total , la media por columnas o por filas , multiplicación escalar a escalar, etc… . Esto hace que el código sea más compacto y fácil de seguir a la vez que más preciso y cómodo de utilizar.

2.2 Representación de las soluciones

Para este problema las soluciones se representan por vectores de la biblioteca Eigen con tipo de dato double donde cada posición del vector representa la i-ésima empresa del fichero de datos de entrada , todos estos vectores deben cumplir las siguientes restricciones:

- El tamaño de la solución debe ser el mismo que el número de empresas en los datos de entrada.
- La suma de todos sus valores debe ser igual a 1.
- Cada posición del vector tiene que tener un valor entre **low** y **high**
- Puede haber posiciones del vector con valor 0(representando que en esa empresa no se invierte).

La mejor solución será aquella que consiga maximizar la función objetivo.

2.3 Funciones comunes usadas por los algoritmos

En primer lugar la función que permite cargar los datos en nuestra clase Pportfolio es una función implementada en un archivo auxiliar ya que no tiene por qué pertenecer necesariamente al problema que se plantea.Esta función lee datos de un fichero en formato CSV. Tras abrir el archivo la función descarta la primera linea con los nombre de las empresas y comienza a leer las siguientes lineas, una vez escogida una linea , se salta el string que contiene la fecha y comienza a insertar datos al vector usando como separador

la coma. El código comprueba los valores de la fecha leida , si el parametro only2025 es True , solo se leen datos de 2025 , de lo contrario se leen todos los datos hasta el último día del 2024. Una vez leidos todos los datos, la matriz obtenida se convierte a una matriz Eigen para poder manejarla con más facilidad.

```
funcion LoadCSV(archivo, only2025):
  matrix data
  leer(primera_linea)
  abrir archivo
  while(leer(linea_archivo)):
    if es fecha el primer string :
      continue
    if(es fecha el primer string and not only2025):
      solo leer valores de 2014-2024
    if(es fecha el primer string and only2025):
      solo leer valores de 2025
    vector valores

    while(leer(dato)):
      if(dato no vacio):
        valores.pushback(dato)
      else:
        valores.pushback(0.0)

    data.pushback(valores)

  cerrar archivo

  if(data no vacio):
    matriz_eigen = data
  return matriz_eigen
```

Las principales funciones que se usan de la clase **Pportfolio** para implementar los algoritmos son las siguientes :

La función **createSolution** Se usa en dos algoritmos , en la Búsqueda Aleatoria para generar soluciones aleatorias y escoger finalmente la que mejor fitness tenga y en la Búsqueda Local para generar la solución desde la cual se comienza a buscar la mejor solución posible.

```
funcion createSolution():
  vector solucion;

  for i<solucion.size :
    solucion(i) = Aleatorio entre high y low


  numeroceros = Aleatorio entre 1 y solucion.size-1
  vectorindices = {0,1,2, ......,solucion.size-1}
  desordenar(vectorindices)
   ## Seleccionamos valores aleatorios que ponemos a 0
  for i<solucion.size:
    solucion(vectorindices[i])=0
```

```
   if la solucion no es valida
     fix(solucion)

   return solution;
```

```
# Pseudocódigo del calculo del beneficio
 funcion profit(solution):

   logdata = log((data+1)) # la operación se aplica a todos las componentes
   return (solution * logdata.colwise.sum).sum #multiplicación componente a
componente
```

```
# Pseudocódigo del calculo driesgo
 funcion risk(solution):
   return (solution.transpose * covarianceMat * solution);
#devuelve un número resultado de las dimensiones  1xn * nxn *nx1
```

```
# Pseudocódigo del calculo del fitness
 funcion fitness(solution):
   return (profit - lambda*risk);
#devuelve un número resultado de las dimensiones  1xn * nxn *nx1
```

```
# Pseudocódigo del calculo de la heurística
 funcion heuristic(i):
   datalogaritmo = (data.columna(i)+1).log().sum()
   mediacovarianza = matrizcovarianza.columna(i).sum()/ncompany
   return datalogaritmo - (lambda * mediacovarianza);
```

El principal objetivo de los algoritmos es maximizar la función fitness , la cual trata de maximizar el beneficio y minimizar el riesgo en una sola fórmula.El parámetro lambda se usará a modo de ajuste , cuando más alto sea su valor el algoritmo valorará mejor soluciones con poco beneficio pero que sean seguras , es decir que tengan poco riesgo , mientras que cuanto más cercano a 0 sea lambda , el algoritmo valorará mejor las soluciones con máximo beneficio posible , ignorando el riesgo.

3 Estructura de los algoritmos

3.1 Random Search

El algoritmo de Búsqueda aleatoria consiste en generar un número determinado de soluciones válidas de forma aleatoria y seleccionar la solución que maximice o minimice la función del problema, en este caso en concreto , se selecciona la solución que maximice el fitness del problema.

```
# Pseudocódigo del algoritmo Random Search
función RandomSearch(max_evals):
   mejorfitnes = -infinito
    for 0 hasta max_evals:
     soluciontemporal = solucion_aleatoria
     if fitnes_soluciontemporal > mejorfitness:
            solucion = mejorsolucion
           fitness =mejorfitness
   return (solucion,fitness,max_evals)
```

Como se puede observar , el algoritmo de Búsqueda aleatoria dependerá de la suerte que tengamos a la hora de generar soluciones aleatorias ya que el espacio de búsqueda de la solución son todas las soluciones posibles a generar en el intervalo dado de número de evaluaciones por tanto no podemos esperar que obtenga una solución que sea mejor que la de otros algoritmos más avanzados , en caso de que esto ocurriese sería por una cuestión de suerte.

3.2 Greedy

El algorítmo Greedy trata de construir una solución ordenando las empresas de mayor a menor según la heurística establecida en el guión . Una vez obtenidas las empresas ordenadas selecciona por orden descendente y construye una solución seleccionado en cada iteración la mejor opción disponible según la heuristica hasta completar el total de la inversión (llegar a suma 1).Como en cada iteración se ha seleccionado la mejor opción posible, el algoritmo intentará asignar el mayor porcentaje posible a esta empresa siempre que le sea posible , de lo contrario , le asignará el porcentaje restante para llegar al cien por cien .

La heurística usada en el problema es la siguiente :

$$heur(i) = \left(\sum_t \log(1 + \mu_{it}) \right) - \lambda \cdot \frac{\sum_j \Sigma_{ij}}{|N|}$$

```
# Pseudocódigo del calculo de la heurística
 funcion heuristic(i):
   datalogaritmo = (data.columna(i)+1).log.sum
   mediacovarianza = matrizcovarianza.columna(i).sum/ncompany
   return datalogaritmo - (lambda * mediacovarianza);
```

```
# Pseudocódigo del algoritmo Greedy
función greedy(maxevals):
  mejorescompanias = vector{0,1,2,....,ncompany-1}
  ordenar por heuristica de mayor a menor -> mejorescompanias
  mejorsolucion = vector{0,0...0}
  while mejorescompanias.sum<1:
    compania = mejorescompanias.front
    mejorescompanias = mejorescompanias.delete(compania)
    mejorsolucion[compania] = minimo entre high y (1-mejorsolucion.sum)

  return (mejorsolucion , fitness(mejorsolucion),1)
```

Podemos ver que el algoritmo greedy , selecciona

3.3 Local Search

Parte de una solución inicial (Greedy o Random Search) y mejora localmente moviendo pequeñas cantidades de peso entre dos activos. Solo se acepta el movimiento si aumenta el fitness; el proceso se detiene cuando no hay mejoras o se agota el presupuesto de evaluaciones.

```
# Pseudocódigo del algoritmo de búsqueda local
función LocalSearch(sol_inicial, max_evals, delta):
    actual = sol_inicial
    mejor = sol_inicial
    evals = 0
    mientras evals < max_evals:
        i, j = elegir_dos_activos()
        candidata = mover_peso(actual, i, j, delta)
        evals += 1
        si candidata es factible y f(candidata) > f(mejor):
            mejor = candidata
            actual = candidata
        si no hay mejora en varias iteraciones:
            romper
    devolver mejor
```

3.4 Metodología

- Preparación de datos: cálculo de μ y Σ con retornos diarios, limpieza de valores faltantes y normalización temporal para evitar sesgos por escalas.
- Configuración experimental: mismo número máximo de evaluaciones por heurística, λ fijado tras pruebas preliminares y tamaño de paso `paso` o `delta` calibrados para equilibrar exploración y rapidez.
- Métricas de calidad: valor de la función objetivo, retorno medio anualizado y volatilidad anualizada; se revisa además el reparto de pesos para asegurar diversificación.
- Validación: división temporal (train 2015-2023, validación 2024, test 2025) para comprobar si las carteras generalizan y no sobreajustan al histórico.

4 Conclusión

El documento está listo para crecer con un estilo profesional y académico.